

Lab-3-program

```
#include <stdio.h>

void linearSearch( int m, int n, int arr[m][n], int key );
void print( int m, int n, int arr[m][n] );
void minMax( int m, int n, int arr[m][n] );
void sum( int m, int n, int arr[m][n] );
void upperTriangular( int m, int n, int arr[m][n] );
void diagonal( int m, int n, int arr[m][n] );
void lowerTriangular( int m, int n, int arr[m][n] );
void sumOfColumns( int m, int n, int arr[m][n] );

int main()
{
    // taking 2D array input
    int m;
    printf("Enter size of square matrix: ");
    scanf("%d", &m);
    int n = m;
    int arr[m][n];

    // taking in elements
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("Enter element %i,%i : ", i, j);
            scanf("%d", &arr[i][j]);
        }
    }

    // printing the 2D array
    print(m, n, arr);

    //1. Linear Search 2D arrays
    int key;
    printf("Enter key to search: ");
    scanf("%d", &key);
    linearSearch(m, n, arr, key);

    // min and max
    minMax(m, n, arr);

    // sum of all elements
    sum(m, n, arr);

    // upper triangular
    printf("Upper triangular elements: \n");
    upperTriangular(m, n, arr);

    // primary and secondary diagonals
    diagonal(m, n, arr);
```

```

// lower triangular
lowerTriangular(m, n, arr);

// sum of sum of columns
sumOfColumns(m, n, arr);
}

void linearSearch( int m, int n, int arr[m][n], int key )
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (key == arr[i][j])
            {
                printf("key found at index %i,%i\n", i, j);
                return;
            }
        }
    }
    printf("Not found!\n");
}

void print( int m, int n, int arr[m][n] )
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            printf("%i ", arr[i][j]);
        }
        printf("\n");
    }
}

void minMax( int m, int n, int arr[m][n] )
{
    int min = arr[0][0];
    int max = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if ( arr[i][j] > max)
            {
                max = arr[i][j];
            }
            if ( arr[i][j] < min)
            {
                min = arr[i][j];
            }
        }
    }
    printf("Max: %i\n", max);
}

```

```

    printf("Min: %i\n", min);
}

void sum( int m, int n, int arr[m][n] )
{
    int sum = 0;
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            sum += arr[i][j];
        }
    }
    printf("Sum of all elements: %i\n", sum);
}

void upperTriangular( int m, int n, int arr[m][n] )
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i <= j)
            {
                printf("%i ", arr[i][j]);
            }
        }
        printf("\n");
    }
}

void diagonal( int m, int n, int arr[m][n] )
{
    printf("Primary diagonal:\n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i == j)
            {
                printf("%i ", arr[i][j]);
            }
        }
        printf("\n");
    }
    printf("Secondary diagonal:\n");
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if ((i + j) == (m - 1))
            {
                printf("%i ", arr[i][j]);
            }
        }
    }
}

```

```

    }
    printf("\n");
}
}

// lower triangular
void lowerTriangular( int m, int n, int arr[m][n] )
{
    for (int i = 0; i < m; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i >= j)
            {
                printf("%i ", arr[i][j]);
            }
        }
        printf("\n");
    }
}

// sum of every column
void sumOfColumns( int m, int n, int arr[m][n] )
{
    for (int i = 0; i < m; i++)
    {
        int sum = 0;
        for (int j = 0; j < n; j++)
        {
            sum += arr[j][i];
        }
        printf("Sum of column %i: %i\n", (i + 1), sum);
    }
}

```

Output

```

Enter size of square matrix: 3
Enter element 0,0 : 1
Enter element 0,1 : 2
Enter element 0,2 : 3
Enter element 1,0 : 4
Enter element 1,1 : 5
Enter element 1,2 : 6
Enter element 2,0 : 7
Enter element 2,1 : 8
Enter element 2,2 : 9
1 2 3
4 5 6
7 8 9
Enter key to search: 5
key found at index 1,1

```

```
Max: 9
Min: 1
Sum of all elements: 45
Upper triangular elements:
1 2 3
5 6
9
Primary diagonal:
1
5
9
Secondary diagonal:
3
5
7
1
4 5
7 8 9
Sum of column 1: 12
Sum of column 2: 15
Sum of column 3: 18
```

Hackerrank

Compare the triplets

```
int* compareTriplets(int a_count, int* a, int b_count, int* b, int* result_count) {
    int *r = malloc(2 * sizeof(int));
    int score_a = 0;
    int score_b = 0;

    for (int i = 0; i < a_count; i++)
    {
        if (a[i] > b[i])
        {
            score_a++;
        }
        else if (a[i] < b[i])
        {
            score_b++;
        }
    }

    r[0] = score_a;
    r[1] = score_b;

    *result_count = 2;
    return r;
}
```

HackerRank
All Contests > DSA Lab 3 Session 2 > Compare the Triplets
Exit Full Screen View

Problem

Finally, for elements 2, $a[2] > b[2]$ so Alice receives a point.

The return array is [1, 1] with Alice's score first and Bob's second.

Function Description

Complete the function compareTriplets with the following parameter(s):

- `int a[3]:` Alice's challenge rating
- `int b[3]:` Bob's challenge rating

Returns

- `int[2]:` the first element is Alice's score and the second is Bob's score

Input Format

The first line contains 3 space-separated integers, $a[0]$, $a[1]$, and $a[2]$, the respective values in triplet a .

The second line contains 3 space-separated integers, $b[0]$, $b[1]$, and $b[2]$, the respective values in triplet b .

Constraints

- $1 \leq a[i] \leq 100$
- $1 \leq b[i] \leq 100$

Sample Input 0

```
5 6 7
3 6 10
```

Sample Output 0

```
1 1
```

Explanation 0

In this example:

- $a = (a[0], a[1], a[2]) = (5, 6, 7)$
- $b = (b[0], b[1], b[2]) = (3, 6, 10)$

Now, let's compare each individual score:

- $a[0] > b[0]$ so Alice receives 1 point.

```

55 int* compareTriplets(int* a_count, int* a, int* b_count, int* b, int* result_count) {
56     int* r = malloc(2 * sizeof(int));
57
58     int score_a = 0;
59     int score_b = 0;
60
61     for (int i = 0; i < a_count; i++)
62     {
63         if (a[i] > b[i])
64         {
65             score_a++;
66         }
67         else if (a[i] < b[i])
68         {
69             score_b++;
70         }
71     }
72 }
```

Line: 71 Col: 6

Upload Code as File
Test against custom input

Run Code
Submit Code

Congratulations

You solved this challenge. Would you like to challenge your friends? [🐦](#) [in](#)

✔ Test case 0

✔ Test case 1

✔ Test case 2

Compiler Message

Success

Activate Windows
Go to Settings to activate Windows.

```
void miniMaxSum(int arr_count, int* arr) {
    long int min = arr[0];
    long int max = arr[0];
    long int minSum = 0;
    long int maxSum = 0;
    long int tempsum = 0;

    for (int i = 1; i < arr_count; i++)
    {
        if (arr[i] > max)
        {
            max = arr[i];
        }
        if (arr[i] < min)
        {
            min = arr[i];
        }
    }

    for (int i = 0; i < arr_count; i++)
    {
        tempsum += arr[i];
    }

    minSum = tempsum - max;
    maxSum = tempsum - min;
    printf("%ld %ld", minSum, maxSum);
}
```

Problem

Complete the `miniMaxSum` function with the following parameter(s):

- `arr[5]`: an array of 5 integers